

Full Stack Hackathon

Expense Tracker — Final Round

[Interviewer Guide](#) · [Process Documentation](#) · [Q&A Answer Key](#)

PROJECT

Expense Tracker

STACK

React + Java Spring Boot

BUILD

2 Hours 30 Minutes

Q&A

5 Questions · 2–3 min each

PASSWORD

Neutara@2026

ADMIN

Admin@Neutara2026

Hackathon Process Overview

Phase 1 — Candidate Logs In [< 2 min]

- URL: <http://yourserver:5000>
- Enter any valid email address
- Password: Neutara@2026
- Lands directly on the Build Phase screen

Phase 2 — Build Phase — Expense Tracker [2 hrs 30 min]

- Candidate sees full project spec: features, API endpoints, stack
- They build on their own machine (Spring Boot + React)
- Live countdown timer shown — camera is ON throughout
- Three tabs available: Features / API Endpoints / Tips & Evaluation
- Timer turns amber at 30 min remaining, red at 15 min

Phase 3 — Code Submission (Required) [5 min]

- Candidate zips their project (must exclude target/ and node_modules/)
- Uploads the .zip file directly in the app (max 100 MB)
- Cannot proceed to Q&A until upload is confirmed
- File saved to: server/uploads/hackathon-code/
- You can download from admin panel at any time

Phase 4 — Video Q&A — 5 Questions [~25 min]

- Camera + microphone required — live video recording
- Each question displayed with topic badge and difficulty
- Up to 3 minutes per question (auto-stops at limit)
- Candidate can re-record before saving and moving on
- Must record a response before clicking Next

Phase 5 — Submission & Results [< 1 min]

- Candidate submits after recording all 5 answers
- Thank-you screen: "We will get back to you shortly"
- Interviewer reviews code via admin panel
- Admin URL: <http://yourserver:5000/admin>

Project Specification

Shown to candidate during build phase

Tech Stack (Required)

Frontend: React.js — functional components, hooks (useState, useEffect)
Backend: Java — Spring Boot (REST API with Spring MVC)
Database: H2 In-Memory / MySQL / PostgreSQL (candidate's choice)
Build Tool: Maven or Gradle

Features to Build

- 1 **Add Expense** — Form: Amount, Category (Food/Travel/Shopping/Bills/Other), Description, Date
- 2 **View Expenses** — Table or list with category colour tags; total displayed at the bottom
- 3 **Edit & Delete** — Update expense via modal/inline form; delete with confirmation dialog
- 4 **Filter Expenses** — Filter by category and/or date range; results update dynamically
- 5 **Dashboard Summary** — Total spent + category breakdown (text required; chart is bonus)

Required REST API Endpoints

GET	/api/expenses	Fetch all expenses (returns JSON array)
POST	/api/expenses	Create a new expense (@RequestBody Expense)
PUT	/api/expenses/{id}	Update an existing expense by ID
DELETE	/api/expenses/{id}	Delete an expense by ID
GET	/api/expenses/summary	Return category-wise total amounts

Evaluation Criteria

Core	Working CRUD operations via REST API
Core	Spring Boot layering: Controller !' Service !' Repository (@Entity, JPA)
Core	React component structure and state management
Core	Code readability, naming conventions, organisation
Bonus	Input validation (Bean Validation / React form validation)
Bonus	Error handling (try/catch, HTTP status codes, UI feedback)
Bonus	Responsive UI, category charts (Recharts / Chart.js)

Walk us through your Expense Tracker project architecture. How does data flow from your React frontend all the way to your Java Spring Boot backend and back?

HINT TO CANDIDATE: Cover: React component !' API call !' Spring Controller !' Service !' Repository !' DB !' response back to UI

EXPECTED ANSWER — INTERVIEWER GUIDE

FRONTEND SIDE

- React component uses `useState` to store expenses array and loading/error state
- `useEffect` triggers a `fetch/axios GET` call to `/api/expenses` on mount
- **HTTP request hits Spring Boot server (typically `http://localhost:8080`)**
 - CORS must be configured: `@CrossOrigin` on controller or `WebMvcConfigurer` bean

BACKEND SIDE (Spring MVC flow)

- `@RestController` receives the HTTP request and routes it via `@GetMapping`
- Controller delegates to `@Service` layer — no business logic in controller
- **Service calls `@Repository` (interface extending `JpaRepository<Expense, Long>`)**
 - JPA/Hibernate translates the repository call to a SQL query on H2/MySQL
 - DB returns rows !' Hibernate maps to `List<Expense>` Java objects

RESPONSE FLOW

- Service returns list to Controller !' Jackson serialises to JSON automatically
- JSON response received by `fetch/axios` in React
- React `setState` updates the expenses array !' component re-renders with new data

WHAT TO LISTEN FOR

- Mentions `@RestController`, `@Service`, `@Repository` annotations
- Understands separation of concerns across layers
- Knows CORS needs configuration for cross-origin calls
- Understands Jackson auto-serialises Java objects to JSON
- Red flag: Says logic lives directly in controller or doesn't mention layers

Explain the REST API endpoints you created in your Spring Boot backend. How did you structure your Controller, Service, and Repository layers, and why?

HINT TO CANDIDATE: Cover: @RestController, @Service, @Repository, JPA/Hibernate, the 5 endpoints

EXPECTED ANSWER — INTERVIEWER GUIDE

ENTITY (Model Layer)

- @Entity class Expense with: @Id @GeneratedValue Long id, String category, Double amount, String description, LocalDate date
- @Table(name='expenses') optional for custom naming

REPOSITORY

- interface ExpenseRepository extends JpaRepository<Expense, Long>
- **Gets CRUD for free: findAll(), save(), findById(), deleteById()**
- Custom: List<Expense> findByCategory(String cat) — Spring auto-implements

SERVICE (@Service)

- Injects repository via constructor injection (preferred) or @Autowired
- **Methods: getAllExpenses(), createExpense(), updateExpense(), deleteExpense()**
- getSummary(): groups by category and sums amounts
- **Business rules live here (e.g. amount must be positive)**

CONTROLLER (@RestController)

- @RequestMapping('/api/expenses') on class
- **GET /api/expenses !' @GetMapping !' getAllExpenses()**
- **POST /api/expenses !' @PostMapping !' createExpense(@RequestBody)**
- **PUT /api/expenses/{id} !' @PutMapping !' updateExpense(@PathVariable, @RequestBody)**
- **DELETE /api/expenses/{id} !' @DeleteMapping !' deleteExpense(@PathVariable)**
- **GET /api/expenses/summary !' @GetMapping !' getSummary()**
- **Returns ResponseEntity<> for proper HTTP status codes (200, 201, 404)**

APPLICATION.PROPERTIES (H2 example)

- spring.datasource.url=jdbc:h2:mem:expensedb
- spring.h2.console.enabled=true
- spring.jpa.hibernate.ddl-auto=create-drop

WHAT TO LISTEN FOR

- Correct annotation usage (@RestController, not @Controller)
- Constructor injection preferred over field injection
- Returns ResponseEntity with appropriate status codes
- Knows JpaRepository eliminates boilerplate SQL
- Red flag: All logic in controller, no service layer

How did you manage state and handle API communication in your React frontend? Walk us through your component structure and how you handled loading and error states.

HINT TO CANDIDATE: Cover: `useState`, `useEffect`, `fetch/axios`, component hierarchy, loading/error handling

EXPECTED ANSWER — INTERVIEWER GUIDE

STATE MANAGEMENT

- `const [expenses, setExpenses] = useState([])` // list of all expenses
- `const [loading, setLoading] = useState(false)` // loading indicator
- `const [error, setError] = useState(null)` // error message
- `const [filter, setFilter] = useState({category:"", dateFrom:"", dateTo:"})`
- `const [editTarget, setEdit] = useState(null)` // expense being edited
- Form state: `{amount:"", category:"", description:"", date:"}`

DATA FETCHING (`useEffect`)

- `useEffect(() => { fetchExpenses(); }, [])` // runs on mount
- `async fetchExpenses() { setLoading(true); try { ... } catch { setError(...)} finally { setLoading(false) } }`
- POST/PUT/DELETE: optimistic update or re-fetch after response

COMPONENT HIERARCHY

- `App.jsx` — top-level state, passes down props/callbacks
- `% % FilterBar` — dropdowns for category & date range
- `% % Dashboard` — summary cards / chart
- `% % AddExpenseForm` — controlled form with validation
- `% % ExpenseTable` — renders expenses array
- `% % % ExpenseRow` — single row with Edit / Delete buttons
- `% % EditModal` — modal/inline form for updating

ERROR & LOADING HANDLING

- `if (loading) return <Spinner />`
- `if (error) return <ErrorMessage message={error} />`
- Empty state: `expenses.length === 0` ! show 'No expenses yet'
- Delete: `window.confirm()` before API call

CORS

- `fetch('http://localhost:8080/api/expenses')` — direct call in development
- Or proxy in `package.json`: `{ 'proxy': 'http://localhost:8080' } !` `fetch('/api/expenses')`

WHAT TO LISTEN FOR

- Mentions `useState` and `useEffect` correctly
- Separates concerns across components (not one giant `App.jsx`)
- Handles loading and error states explicitly
- Understands controlled inputs (value + `onChange`)
- Red flag: Mutates state directly, no error handling, everything in one component

What was the most challenging part of building this Expense Tracker within the time limit? How did you debug and solve it, and what would you improve with more time?

HINT TO CANDIDATE: Looking for: specific problem, debugging approach, concrete future improvements

EXPECTED ANSWER — INTERVIEWER GUIDE

COMMON GENUINE CHALLENGES (any of these shows real experience)

- CORS errors when React calls Spring Boot on a different port
! Fixed by @CrossOrigin or adding proxy in package.json
- JPA entity not saving correctly (missing @Entity or wrong column type)
! Debugged via H2 console (<http://localhost:8080/h2-console>)
- LocalDate serialisation issue (Jackson can't serialise by default)
! Added @JsonFormat or jackson-datatype-jsr310 dependency
- React state not updating after delete (mutating array instead of filter)
! Fixed: `setExpenses(prev => prev.filter(e => e.id !== id))`
- Date format mismatch between React input (string) and Java (LocalDate)
! Used @DateTimeFormat or parsed manually

DEBUGGING APPROACH (what good candidates mention)

- Spring Boot: checked console logs, used Postman to test API independently
- React: browser DevTools (Network tab to inspect requests), console.log state
- H2 console to verify data was actually being persisted
- Isolated frontend vs backend to find where the issue originated

IMPROVEMENTS WITH MORE TIME

- JWT-based user authentication (each user sees only their expenses)
- Pagination (large expense lists)
- Charts (Recharts or Chart.js for category pie chart / monthly bar chart)
- Input validation (Bean Validation @NotNull, @Min on backend; Formik on frontend)
- Unit tests (JUnit + Mockito for service layer; React Testing Library)
 - Export to CSV / PDF
 - Recurring expense templates
 - Budget limits with alerts

SCORING THIS ANSWER

- Excellent: Names a specific real problem with root cause and exact fix
- Good: Describes debugging process (Postman, H2 console, DevTools)
- Good: Lists meaningful improvements, not just 'better UI'
- Weak: 'Everything went smoothly' — suspicious for a 2.5-hr project
- Weak: Vague challenges with no technical detail

How would you add JWT-based user authentication to this Expense Tracker? Walk us through the exact changes needed in your Spring Boot backend using Spring Security, and the changes on the React side.

HINT TO CANDIDATE: Cover: Spring Security config, JWT filter, /auth/login endpoint, React localStorage, Authorization header

EXPECTED ANSWER — INTERVIEWER GUIDE

BACKEND — pom.xml DEPENDENCIES

- spring-boot-starter-security
- io.jsonwebtoken:jjwt-api, jjwt-impl, jjwt-jackson (or java-jwt by Auth0)

BACKEND — SECURITY CONFIG

- @Configuration @EnableWebSecurity class SecurityConfig
- SecurityFilterChain bean:
 - csrf().disable() (REST API, no form login)
 - sessionManagement: STATELESS
 - authorizeRequests: permitAll for /auth/**, authenticated for /api/**
 - addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)

BACKEND — JWT FILTER

- class JwtFilter extends OncePerRequestFilter
- doFilterInternal(): extract 'Authorization: Bearer <token>' header
- Validate token with JwtUtil (secret key, expiry check)
 - Extract username, load UserDetails, set SecurityContextHolder

BACKEND — AUTH ENDPOINTS

- @RestController @RequestMapping('/auth')
- POST /auth/register !' save User (BCrypt hashed password)
 - POST /auth/login !' authenticate, return { token: '...', email: '...' }
 - User entity: id, email, password (BCrypt), role

BACKEND — DATA CHANGE

- Expense entity gets @ManyToOne User owner field
- GET /api/expenses filters by logged-in user: repo.findByOwner(currentUser)
 - POST creates expense with owner = currentUser from SecurityContext

REACT SIDE

- After POST /auth/login: localStorage.setItem('token', data.token)
 - All API calls: headers: { Authorization: `Bearer \${localStorage.getItem('token')}` }
 - Create PrivateRoute component: if no token !' redirect to /login
 - Logout: localStorage.removeItem('token') !' redirect to /login
 - Axios interceptor (optional): auto-attach token to every request

SCORING THIS ANSWER

- Excellent: Covers SecurityConfig, JwtFilter, filter chain order, React header
- Good: Knows STATELESS session, CSRF disabled for REST, BCrypt for passwords
- Good: Mentions extracting user from SecurityContext for data isolation
- Weak: Only mentions 'add Spring Security' without explaining configuration
- Red flag: Stores password in plain text or stores JWT in sessionStorage without awareness of XSS

Admin Review Guide

Accessing the Admin Panel

- Open a browser and go to: `http://localhost:5000/admin` (or your server's IP/domain)
- Enter the admin password: `Admin@Neutara2026`
- All candidate submissions appear — newest first
- Each row shows: email, submission time, file size, and original filename
- Click "Download Code" to get their project .zip file
- Extract the zip and open in IntelliJ IDEA or VS Code to review the code

Code Review Checklist

- Spring Boot project structure: `src/main/java` with `controller/`, `service/`, `repository/`, `model/` packages
- Entity class: correct `@Entity`, `@Id`, `@GeneratedValue` annotations; appropriate field types
- `JpaRepository` used (not raw `SQL/JDBC` unless explicitly chosen)
- Controller uses `@RestController`; endpoints match `GET/POST/PUT/DELETE/summary` spec
- Service layer handles business logic; controller is thin
- CORS configured (not just `@CrossOrigin` on controller — check if properly scoped)
- `application.properties/yml`: DB config present; H2 console enabled for in-memory
- React components separated (not a monolithic `App.jsx`)
- API calls use `fetch` or `axios`; `async/await` or `.then()` chains; error handling present
- Filter feature works client-side or via query params to backend
- Category summary calculated correctly
- Bonus: input validation, responsive CSS, chart library integrated

Quick Reference — Passwords & URLs

Candidate login URL: `http://localhost:5000` (or your deployed domain)

Candidate password: `Neutara@2026`

Admin panel URL: `http://localhost:5000/admin`

Admin password: `Admin@Neutara2026`

Code upload folder: `server/uploads/hackathon-code/` (on your server)

Submissions log: `server/data/hackathon-submissions.json`

Q&A questions file: `server/data/hackathon-qa-questions.json`

Scoring Rubric (Suggested)

Code Quality: 25% — Clean architecture, separation of concerns, readable code

API Design: 25% — Correct REST endpoints, proper HTTP methods and status codes

Feature Completeness: 20% — All 5 features working: CRUD, filter, summary

Q&A Depth: 20% — Technical accuracy and clarity in video answers

Problem Solving: 10% — How they handled challenges, what they'd improve

